

C# Lab Exercise: Refactoring a Real-World Application Using SOLID Principles

Scenario

You are working on an **E-Commerce Order Processing System** for an online shopping company.

A junior developer implemented everything quickly in a single class. The application works, but now:

- New payment methods are difficult to add
- Logging changes require modifying core business logic
- Email/SMS notifications are tightly coupled
- Reports are difficult to extend
- Testing is painful

Your task is to identify the SOLID principle violations and refactor the code step-by-step.

Lab Objectives

By the end of this lab, you will learn:

- How to identify violations of SOLID principles
 - How tightly coupled code creates maintenance problems
 - How to refactor code using:
 - SRP
 - OCP
 - LSP
 - ISP
 - DIP
 - How to design extensible enterprise applications
-

Initial Wrong Design

Problem Statement

The company currently supports:

- Order placement
- Payment processing
- Invoice generation
- Email notifications
- Logging

Everything is implemented in one class.

Step 1 — Analyze the Existing Bad Code

```
using System;

public class OrderProcessor
{
    public void ProcessOrder(string customer, double amount, string
paymentType)
    {
        Console.WriteLine("Order processing started...");

        // Payment Processing
        if (paymentType == "CreditCard")
        {
            Console.WriteLine("Processing credit card payment...");
        }
        else if (paymentType == "UPI")
        {
            Console.WriteLine("Processing UPI payment...");
        }
        else if (paymentType == "NetBanking")
        {
            Console.WriteLine("Processing NetBanking payment...");
        }

        // Generate Invoice
        Console.WriteLine("Generating invoice...");

        // Send Notification
        Console.WriteLine($"Sending email to {customer}");

        // Logging
        Console.WriteLine("Logging order details into file");

        Console.WriteLine("Order processed successfully");
    }
}
```

Step 2 — Identify SOLID Violations

1. SRP Violation (Single Responsibility Principle)

Problem

The class is handling:

- Order processing
- Payment processing
- Invoice generation
- Notification
- Logging

A class should have only ONE reason to change.

Why is this bad?

If tomorrow:

- Logging changes
- Email system changes
- Invoice format changes

You must modify the same class.

This increases:

- Bugs
 - Regression risk
 - Maintenance complexity
-

2. OCP Violation (Open/Closed Principle)

Problem

```
if (paymentType == "CreditCard")
```

Every new payment type requires modifying existing code.

Example:

- Add Wallet payment
- Add Crypto payment
- Add EMI payment

The class keeps changing.

3. DIP Violation (Dependency Inversion Principle)

The class directly depends on low-level implementations:

```
Console.WriteLine("Sending email...");
```

No abstraction exists.

High-level modules should depend on abstractions.

Step 3 — Refactor Using SOLID Principles

PART A — Fix SRP

Create Separate Responsibilities

Payment Service

```
public class PaymentService
{
    public void ProcessPayment(string paymentType, double amount)
    {
        Console.WriteLine($"Processing {paymentType} payment of {amount}");
    }
}
```

Invoice Service

```
public class InvoiceService
{
    public void GenerateInvoice(double amount)
    {
        Console.WriteLine($"Invoice generated for {amount}");
    }
}
```

Notification Service

```
public class NotificationService
{
    public void SendEmail(string customer)
    {
        Console.WriteLine($"Email sent to {customer}");
    }
}
```

Logging Service

```
public class LoggingService
{
    public void Log(string message)
    {
        Console.WriteLine($"LOG: {message}");
    }
}
```

Updated Order Processor

```
public class OrderProcessor
{
    private readonly PaymentService _paymentService;
    private readonly InvoiceService _invoiceService;
    private readonly NotificationService _notificationService;
    private readonly LoggingService _loggingService;

    public OrderProcessor()
    {
        _paymentService = new PaymentService();
        _invoiceService = new InvoiceService();
        _notificationService = new NotificationService();
        _loggingService = new LoggingService();
    }

    public void ProcessOrder(string customer, double amount, string paymentType)
    {

```

```
        _loggingService.Log("Order started");  
        _paymentService.ProcessPayment(paymentType, amount);  
        _invoiceService.GenerateInvoice(amount);  
        _notificationService.SendEmail(customer);  
        _loggingService.Log("Order completed");  
    }  
}
```

Improvement Achieved

Now each class has a single responsibility.

PART B — Fix OCP

Problem

Still using:

if-else

for payment types.

Apply Strategy Pattern

Create Abstraction

```
public interface IPaymentMethod  
{  
    void Pay(double amount);  
}
```

Credit Card Payment

```
public class CreditCardPayment : IPaymentMethod
{
    public void Pay(double amount)
    {
        Console.WriteLine($"Credit Card payment of {amount} processed");
    }
}
```

UPI Payment

```
public class UpiPayment : IPaymentMethod
{
    public void Pay(double amount)
    {
        Console.WriteLine($"UPI payment of {amount} processed");
    }
}
```

NetBanking Payment

```
public class NetBankingPayment : IPaymentMethod
{
    public void Pay(double amount)
    {
        Console.WriteLine($"NetBanking payment of {amount} processed");
    }
}
```

Updated Payment Service

```
public class PaymentService
{
    public void ProcessPayment(IPaymentMethod paymentMethod, double amount)
    {
        paymentMethod.Pay(amount);
    }
}
```

Benefit

Now adding a new payment type does NOT modify existing code.

Just create:

```
public class WalletPayment : IPaymentMethod
```

This follows OCP.

PART C — Fix DIP

Problem

OrderProcessor directly creates dependencies:

```
new PaymentService()
```

Use Dependency Injection

Create Interfaces

```
public interface IInvoiceService
{
    void GenerateInvoice(double amount);
}

public interface INotificationService
{
    void Send(string customer);
}

public interface ILoggerService
{
    void Log(string message);
}
```

Implement Interfaces

```
public class InvoiceService : IInvoiceService
{
    public void GenerateInvoice(double amount)
    {
        Console.WriteLine($"Invoice generated for {amount}");
    }
}
```

Refactored OrderProcessor

```
public class OrderProcessor
{
    private readonly PaymentService _paymentService;
    private readonly IInvoiceService _invoiceService;
```



```

private readonly INotificationService _notificationService;
private readonly ILoggerService _loggerService;

public OrderProcessor(
    PaymentService paymentService,
    IInvoiceService invoiceService,
    INotificationService notificationService,
    ILoggerService loggerService)
{
    _paymentService = paymentService;
    _invoiceService = invoiceService;
    _notificationService = notificationService;
    _loggerService = loggerService;
}

public void ProcessOrder(
    string customer,
    double amount,
    IPaymentMethod paymentMethod)
{
    _loggerService.Log("Order Started");

    _paymentService.ProcessPayment(paymentMethod, amount);

    _invoiceService.GenerateInvoice(amount);

    _notificationService.Send(customer);

    _loggerService.Log("Order Completed");
}
}

```

Benefit

Now:

- Easily testable
 - Loosely coupled
 - Extensible
 - Enterprise-ready
-

PART D — ISP Violation Exercise

Wrong Interface

```

public interface IWorker
{
    void Work();
}

```

```
        void Eat();  
    }  
}
```

Problem

Robot workers do not eat.

```
public class Robot : IWorker  
{  
    public void Work()  
    {  
        Console.WriteLine("Robot working");  
    }  
  
    public void Eat()  
    {  
        throw new NotImplementedException();  
    }  
}
```

ISP Fix

Split Interfaces

```
public interface IWorkable  
{  
    void Work();  
}  
  
public interface IEatable  
{  
    void Eat();  
}
```

Human Worker

```
public class HumanWorker : IWorkable, IEatable  
{  
    public void Work()  
    {  
        Console.WriteLine("Human working");  
    }  
  
    public void Eat()  
    {  
        Console.WriteLine("Human eating");  
    }  
}
```

Robot Worker

```
public class RobotWorker : IWorkable
{
    public void Work()
    {
        Console.WriteLine("Robot working");
    }
}
```

PART E — LSP Violation Exercise

Wrong Design

```
public class Bird
{
    public virtual void Fly()
    {
        Console.WriteLine("Bird flying");
    }
}

public class Ostrich : Bird
{
    public override void Fly()
    {
        throw new NotImplementedException();
    }
}
```

Why LSP Violated?

An Ostrich is not substitutable for Bird.

LSP Fix

Better Abstraction

```
public abstract class Bird
{
    public abstract void Move();
}
```

Flying Bird

```
public class Sparrow : Bird
{
    public override void Move()
    {
        Console.WriteLine("Sparrow flying");
    }
}
```

Non-Flying Bird

```
public class Ostrich : Bird
{
    public override void Move()
    {
        Console.WriteLine("Ostrich running");
    }
}
```

Final Architecture

Classes

- OrderProcessor
- PaymentService
- InvoiceService
- NotificationService
- LoggerService

Interfaces

- IPaymentMethod
- IInvoiceService
- INotificationService
- ILoggerService

Principles Applied

SOLID Principle	Applied Solution
SRP	Split responsibilities

SOLID Principle	Applied Solution
OCP	Strategy Pattern
LSP	Proper inheritance
ISP	Small focused interfaces
DIP	Dependency Injection

Student Tasks

Task 1

Add a new payment method:

`WalletPayment`

WITHOUT modifying existing payment classes.

Task 2

Add SMS notification support.

Requirement:

- Existing code should not change significantly.
-

Task 3

Add database logging instead of console logging.

Use:

`ILoggerService`

Task 4

Unit test `OrderProcessor` using mock services.

Task 5

Convert the application into ASP.NET Core Web API.

Expected Learning Outcomes

Students will understand:

- Why bad design becomes difficult to maintain
- How SOLID improves scalability
- How interfaces reduce coupling
- How design patterns support SOLID
- How enterprise applications are architected